Разработка модулей ядра Linux: Часть 9. Интерфейсы модуля для взаимодействия с пользовательскими процессами

Олег Цилюрик
преподаватель тренингового отделения
Global Logic

17.05.2012

Эта статья продолжает цикл статей, посвященный написанию модулей ядра Linux. В статье будут рассмотрены схемы взаимодействия и интерфейсы, по которым модуль «сотрудничает» с другими компонентами операционной системы Linux.

Введение

Данная статья продолжает изучение интерфейсов, используемых модулями ядра для взаимодействия с другими компонентами платформы Linux. В ней рассматриваются вопросы взаимодействия модуля с пользовательским пространством (пользовательскими приложениями, пространством файловых имён, реальным оборудованием, каналами обмена данными и т.д.).

Взаимодействие модуля ядра с уровнем пользователя

В случае с интерфейсом, используемым модулем для взаимодействия с ядром, всё достаточно понятно и единообразно, хотя объём информации и очень велик. Но для интерфейса, используемого модулем ядра для коммуникаций с уровнем пользователя, складывается абсолютно противоположная ситуация, так как существует множество различных способов взаимодействия, используемых в различных ситуациях.

1. **Вывод диагностической информации из модуля** (в системный журнал) с помощью вызова `printk()`:
   - осуществляет вывод в **текстовую консоль** (не в графический терминал!);
   - осуществляет вывод в файл системного журнала **/var/log/messages**;
   - содержимое файла журнала можно смотреть и изучать при помощи команды `dmesg`;

   Диагностические сообщения, передаваемые из модуля по `printk()` (что бы ни говорилось в публикациях про перспективы диалоговых отладчиков ядра), остаются основным и самым массовым инструментом для **отладки** кода модулей ядра. Также по этому каналу модуль сообщает о серьёзных или критических ошибках в его работе.

© Copyright IBM Corporation 2012

2. **Копирование данных** между пользовательским адресным пространством и пространством ядра. Такие операции требуют привилегированного режима процессора, могут выполняться только кодом ядра, выполняются исключительно по инициативе модуля и никогда по инициативе пользователя. Конечно, это всё те же вызовы из набора API ядра, уже обсуждавшегося выше. Но эти **четыре** вызова несут особую нагрузку, и предназначены для узко утилитарных целей: обмен данными между адресным пространством ядра и пространством пользователя. Без таких операций взаимодействие пользователя с ядром было бы невозможно, также невозможным было бы выполнение системных вызовов и вообще существование пользовательского окружения.
   Эти вызовы перечислены в листинге 4 (сначала они извлекаются из динамической таблицы имён ядра, а затем просматриваются их определения в заголовочных файлах):

Листинг 4. Вызовы для обмена данными

```
$ cat /proc/kallsyms | grep put_user
c05c634c T __put_user_1
c05c6360 T __put_user_2
c05c6378 T __put_user_4
c05c6390 T __put_user_8
...
$ cat /proc/kallsyms | grep get_user
...
c05c628c T __get_user_1
c05c62a0 T __get_user_2
c05c62b8 T __get_user_4
...
$ cat /proc/kallsyms | grep copy_to_user
...
c05c6afa T copy_to_user
...
$ cat /proc/kallsyms | grep copy_from_user
...
c04abfeb T iov_iter_copy_from_user
c04ac053 T iov_iter_copy_from_user_atomic
...
c05c69e1 T copy_from_user
...
```

Вызовы `put_user()` и `get_user()` - это **макросы**, которые пытаются определить размер (именно в силу того, что являются макросами) пересылаемой последовательности данных (1, 2, 4 байта - для `get_user()` и 1, 2, 4, 8 байт - для `put_user()`, причину такой ассиметрии я объяснить не могу). Вызовы `copy_to_user()` и `copy_from_user()` являются **функциями** API ядра для обмена данными произвольного размера, но на самом деле они просто используют в цикле `put_user()` или `get_user()` соответственно необходимое число раз. Определения всех этих API можно найти в файле **<asm/uaccess.h>**, а их прототипы имеют следующий вид (для макросов `put_user()` и `get_user()` восстановлен тот вид, какой они имели бы, если бы были определены как функциональные вызовы: с типизированными параметрами):

Листинг 5. Прототипы вызовов обмена данными

```
long copy_from_user( void *to, const void __user * from, unsigned long n );
long copy_to_user( void __user *to, const void *from, unsigned long n );
int put_user( void *x, void __user *ptr );
int get_user( void *x, const void __user *ptr );
```

Каждый из этих вызовов возвращает реально скопированное число байт или код ошибки операции (отрицательное значение). В комментариях утверждается, что передача коротких порций данных через `put_user()` и `get_user()` будет осуществляться быстрее. Заслуживает внимания тот факт, что все эти вызовы получают два адресных параметра: источник (`from`) и приёмник (`to`), но это будут значения, относящиеся к разным системам координат, т.е. к разным адресным пространствам. В принципе, в каком-то частном случае эти адреса могут численно совпадать, указывая при этом на совершенно разные физические области. Как это работает лучше понять на примерах, которые вскоре будут продемонстрированы.

Совершенно понятно, что такие операции передачи данных между изолированными защищёнными адресными пространствами может выполнять только код, наделённый привилегиями супервизора, так как для этого требуется манипулировать физическими таблицами отображения виртуальных страниц и работать с привилегированными командами и регистрами процессора. Это может делать код ядра, но модуль и есть ядро!

3. **Интерфейс взаимодействия с каталогом устройств**. Это большая группа структур данных и функций, которые обеспечивают модулю возможности по созданию и поддержанию **символьных имён устройств** вида **/dev/XXX**. С помощью этого инструментария модуль может осуществлять стандартные операции ввода-вывода на устройстве (как символьном, там и блочном), и тем самым становится **драйвером устройства**. Это основной интерфейс от модуля к пользовательскому уровню и, пожалуй, самый важный для конечного пользователя. Кроме того, это один из самых старых механизмов в системе Linux. Также через этот интерфейс реализуется парадигма устройств **/dev** во всех UNIX-системах, а не только в системе Linux. В следующих статьях он будет подвергнут самому тщательному исследованию.
4. Взаимодействие через файлы системы **/proc** (**файловая система** procfs). Модуль может создавать специфические для него индикативные псевдофайлы (или даже целые файловые иерархии) в **/proc**, а затем выводить туда отладочную или диагностическую информацию или считывать оттуда управляющую информацию. Эти файлы в **/proc** доступны для чтения-записи всеми стандартными командами Linux (в пределах регламента прав доступа, установленных для конкретного файлового имени). Этот интерфейс также будет детально рассмотрен далее.
5. Взаимодействие через файлы системы **/sys** (**файловая система** sysfs). Эта файловая система подобна (по назначению) **/proc**, но возникла заметно позже, и считается, что её функциональность значительно богаче и по многим аспектам она рассматривается в Linux в качестве замены **/proc**. В отличие от файловых систем **/dev** и **/proc**, файловая система **/sys** не является общим понятием для всех UNIX систем, никак не связана со стандартами POSIX и не поддерживается этими стандартами. Это собственное

"изобретение" разработчиков **ядра** Linux. Эта подсистема также будет рассмотрена с точки зрения, как она видится из кода модуля ядра, но не так подробно, как **/dev** или **/proc**.

6. Взаимодействие модуля со **стеком сетевых протоколов**, главным образом, со стеком протоколов TCP/IP. Ограничение относительно TCP/IP не принципиально, но так как стек протокола IP развит в Linux сильнее, нежели стеки других протоколов, то и используется он чаще, хотя другие протоколы тоже поддерживаются. Эта подсистема будет детально рассмотрена с помощью примеров кода, как отдельный интерфейс для взаимодействия модуля.

Заключение

В этой статье были рассмотрены виды коммуникаций, существующие между модулем ядра и пространством пользователя. Далее различные типы взаимодействий уже можно изучать независимо друг от друга, что и будет сделано в следующих статьях цикла.

Об авторе

Олег Цилюрик



Олег Иванович Цилюрик, много лет был разработчиком программного обеспечения в крупных центрах разработки: ВНИИ РТ, НПО "Дельта", КБ ПМ. Последние годы работал над проектами в области промышленной автоматики, IP телефонии и коммуникаций. Автор нескольких книг. Преподаватель тренингового отделения международной софтверной компании Global Logic.

© Copyright IBM Corporation 2012
(www.ibm.com/legal/copytrade.shtml)
Торговые марки
(www.ibm.com/developerworks/ru/ibm/trademarks/)